

AG1- Actividad Guiada 1

Alfredo Kaleniuszka

<https://colab.research.google.com/drive/1bADd1xKv6-7qE6eBclgBafGOXsvsTtqC?usp=sharing>

<https://github.com/akaleniuszka/03MIAR--Algoritmo-de-Optimizacion/AG1>

Torres de Hanoi con Divide y vencerás

```
In [1]: def Torres_Hanoi(N, desde, hasta):
        if N ==1 :
            print("Lleva la ficha " ,desde , " hasta " , hasta )

        else:
            #Torres_Hanoi(N-1, desde, 6-desde-hasta )
            Torres_Hanoi(N-1, desde, 6-desde-hasta )
            print("Lleva la ficha " ,desde , " hasta " , hasta )
            #Torres_Hanoi(N-1,6-desde-hasta, hasta )
            Torres_Hanoi(N-1, 6-desde-hasta , hasta )

Torres_Hanoi(3, 1 , 3)
```

```
Lleva la ficha  1  hasta  3
Lleva la ficha  1  hasta  2
Lleva la ficha  3  hasta  2
Lleva la ficha  1  hasta  3
Lleva la ficha  2  hasta  1
Lleva la ficha  2  hasta  3
Lleva la ficha  1  hasta  3
```

Sucesión_de_Fibonacci

https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci

Calculo del termino n-simo de la suscesión de Fibonacci

```
In [2]: def Fibonacci(N:int):
        if N < 2:
            return 1
        else:
            return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

Out[2]: 8

Devolución de cambio por técnica voraz

```
In [3]: def cambio_monedas(N, SM):
        SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]
        ValorAcumulado = 0

        for i,valor in enumerate(SM):
            monedas = (N-ValorAcumulado)//valor
            SOLUCION[i] = monedas
            ValorAcumulado = ValorAcumulado + monedas*valor

        if ValorAcumulado == N:
            return SOLUCION

cambio_monedas(15,[25,10,5,1])
```

Out[3]: [0, 1, 1, 0]

N-Reinas por técnica de vueta atrás

```
In [4]: def escribe(S):
        n = len(S)
        for x in range(n):
            print("")
            for i in range(n):
                if S[i] == x+1:
                    print(" X " , end="")
                else:
```

```

        print(" - ", end="")

def es_prometedora(SOLUCION,etapa):
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos reinas en la misma fila
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.count(SOLUCION[i])) + " veces")
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

    #Verifica Las diagonales
    for j in range(i+1, etapa +1 ):
        #print("Comprobando diagonal de " + str(i) + " y " + str(j))
        if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True

def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i

        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
                #escribe(solucion)
                print()
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

        solucion[etapa] = 0

reinas(8)

```

[1, 5, 8, 6, 3, 7, 2, 4]
[1, 6, 8, 3, 7, 4, 2, 5]
[1, 7, 4, 6, 8, 2, 5, 3]
[1, 7, 5, 8, 2, 4, 6, 3]
[2, 4, 6, 8, 3, 1, 7, 5]
[2, 5, 7, 1, 3, 8, 6, 4]
[2, 5, 7, 4, 1, 8, 6, 3]
[2, 6, 1, 7, 4, 8, 3, 5]
[2, 6, 8, 3, 1, 4, 7, 5]
[2, 7, 3, 6, 8, 5, 1, 4]
[2, 7, 5, 8, 1, 4, 6, 3]
[2, 8, 6, 1, 3, 5, 7, 4]
[3, 1, 7, 5, 8, 2, 4, 6]
[3, 5, 2, 8, 1, 7, 4, 6]
[3, 5, 2, 8, 6, 4, 7, 1]
[3, 5, 7, 1, 4, 2, 8, 6]
[3, 5, 8, 4, 1, 7, 2, 6]
[3, 6, 2, 5, 8, 1, 7, 4]
[3, 6, 2, 7, 1, 4, 8, 5]
[3, 6, 2, 7, 5, 1, 8, 4]
[3, 6, 4, 1, 8, 5, 7, 2]
[3, 6, 4, 2, 8, 5, 7, 1]
[3, 6, 8, 1, 4, 7, 5, 2]
[3, 6, 8, 1, 5, 7, 2, 4]
[3, 6, 8, 2, 4, 1, 7, 5]
[3, 7, 2, 8, 5, 1, 4, 6]
[3, 7, 2, 8, 6, 4, 1, 5]
[3, 8, 4, 7, 1, 6, 2, 5]
[4, 1, 5, 8, 2, 7, 3, 6]
[4, 1, 5, 8, 6, 3, 7, 2]
[4, 2, 5, 8, 6, 1, 3, 7]
[4, 2, 7, 3, 6, 8, 1, 5]
[4, 2, 7, 3, 6, 8, 5, 1]
[4, 2, 7, 5, 1, 8, 6, 3]
[4, 2, 8, 5, 7, 1, 3, 6]
[4, 2, 8, 6, 1, 3, 5, 7]
[4, 6, 1, 5, 2, 8, 3, 7]
[4, 6, 8, 2, 7, 1, 3, 5]
[4, 6, 8, 3, 1, 7, 5, 2]
[4, 7, 1, 8, 5, 2, 6, 3]
[4, 7, 3, 8, 2, 5, 1, 6]
[4, 7, 5, 2, 6, 1, 3, 8]
[4, 7, 5, 3, 1, 6, 8, 2]
[4, 8, 1, 3, 6, 2, 7, 5]

[4, 8, 1, 5, 7, 2, 6, 3]
[4, 8, 5, 3, 1, 7, 2, 6]
[5, 1, 4, 6, 8, 2, 7, 3]
[5, 1, 8, 4, 2, 7, 3, 6]
[5, 1, 8, 6, 3, 7, 2, 4]
[5, 2, 4, 6, 8, 3, 1, 7]
[5, 2, 4, 7, 3, 8, 6, 1]
[5, 2, 6, 1, 7, 4, 8, 3]
[5, 2, 8, 1, 4, 7, 3, 6]
[5, 3, 1, 6, 8, 2, 4, 7]
[5, 3, 1, 7, 2, 8, 6, 4]
[5, 3, 8, 4, 7, 1, 6, 2]
[5, 7, 1, 3, 8, 6, 4, 2]
[5, 7, 1, 4, 2, 8, 6, 3]
[5, 7, 2, 4, 8, 1, 3, 6]
[5, 7, 2, 6, 3, 1, 4, 8]
[5, 7, 2, 6, 3, 1, 8, 4]
[5, 7, 4, 1, 3, 8, 6, 2]
[5, 8, 4, 1, 3, 6, 2, 7]
[5, 8, 4, 1, 7, 2, 6, 3]
[6, 1, 5, 2, 8, 3, 7, 4]
[6, 2, 7, 1, 3, 5, 8, 4]
[6, 2, 7, 1, 4, 8, 5, 3]
[6, 3, 1, 7, 5, 8, 2, 4]
[6, 3, 1, 8, 4, 2, 7, 5]
[6, 3, 1, 8, 5, 2, 4, 7]
[6, 3, 5, 7, 1, 4, 2, 8]
[6, 3, 5, 8, 1, 4, 2, 7]
[6, 3, 7, 2, 4, 8, 1, 5]
[6, 3, 7, 2, 8, 5, 1, 4]
[6, 3, 7, 4, 1, 8, 2, 5]
[6, 4, 1, 5, 8, 2, 7, 3]
[6, 4, 2, 8, 5, 7, 1, 3]
[6, 4, 7, 1, 3, 5, 2, 8]
[6, 4, 7, 1, 8, 2, 5, 3]
[6, 8, 2, 4, 1, 7, 5, 3]
[7, 1, 3, 8, 6, 4, 2, 5]
[7, 2, 4, 1, 8, 5, 3, 6]
[7, 2, 6, 3, 1, 4, 8, 5]
[7, 3, 1, 6, 8, 5, 2, 4]
[7, 3, 8, 2, 5, 1, 6, 4]
[7, 4, 2, 5, 8, 1, 3, 6]
[7, 4, 2, 8, 6, 1, 3, 5]

[7, 5, 3, 1, 6, 8, 2, 4]

[8, 2, 4, 1, 7, 5, 3, 6]

[8, 2, 5, 3, 1, 7, 4, 6]

[8, 3, 1, 6, 2, 5, 7, 4]

[8, 4, 1, 3, 6, 2, 7, 5]

Viaje por el rio. Programación dinámica

```
In [5]: TARIFAS = [
[0,5,4,3,999,999,999],
[999,0,999,2,3,999,11],
[999,999, 0,1,999,4,10],
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#####

def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in [9999]*N]
    RUTA = [ [""]*N for i in [""]*N]

    for i in range(0,N-1):
        RUTA[i][i] = i                #Para ir de i a i se "pasa por i"
        PRECIOS[i][i] = 0             #Para ir de i a i se se paga 0
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k          #Anota que para ir de i a j hay que pasar por k
                PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
#####

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
    if desde == hasta:
        #print("Ir a :" + str(desde))
        return ""
    else:
        return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta])) + \
            ',' + \
            str(RUTA[desde][hasta] \
            )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)
```

```
PRECIOS
[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]
```

```
ruta
[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
['', '', '', '', '', '', '']
```

La ruta es:

Out[5]: ',0,2,5'

Instrucciones de la actividad

- Desarrollar algoritmos voraces para resolver problemas
- Desarrollar algoritmos con la técnica de vuelta atrás(backtracking) para resolver problemas
- Desarrollar algoritmos con la técnica de divide y vencerás para resolver problemas

1. Algoritmo voraz: Mochila fraccionaria

1.1 Descripción del problema

El problema de la **mochila fraccionaria** consiste en seleccionar objetos con diferentes pesos y valores para maximizar el valor total transportado, sin superar una capacidad máxima de peso.

A diferencia de la mochila clásica 0/1, en este caso se permite tomar una **fracción** de un objeto. Por eso, una estrategia voraz funciona correctamente.

1.2 Idea del algoritmo

La estrategia consiste en calcular la relación:

$$\text{valor por peso} = \frac{\text{valor}}{\text{peso}}$$

Luego se ordenan los objetos de mayor a menor según esa relación y se van agregando a la mochila mientras haya capacidad disponible.

Este algoritmo es **voraz** porque en cada paso toma la mejor decisión local: elegir el objeto con mayor valor por unidad de peso.

```
In [6]: def mochila_fraccionaria(objetos, capacidad):
        """
        Resuelve el problema de la mochila fraccionaria usando una estrategia voraz.

        Parámetros:
        objetos: lista de diccionarios con nombre, valor y peso.
        capacidad: capacidad máxima de la mochila.

        Retorna:
        valor_total: valor máximo obtenido.
        seleccionados: lista de objetos seleccionados, indicando si se tomó completo o fraccionado.
        """

        # Calculamos la relación valor/peso para cada objeto
        for objeto in objetos:
            objeto["valor_por_peso"] = objeto["valor"] / objeto["peso"]

        # Ordenamos los objetos de mayor a menor valor por peso
        objetos_ordenados = sorted(
            objetos,
            key=lambda x: x["valor_por_peso"],
            reverse=True
        )

        valor_total = 0
        capacidad_restante = capacidad
        seleccionados = []
```



```

for objeto in objetos_ordenados:
    if capacidad_restante == 0:
        break

    # Si el objeto entra completo, se toma completo
    if objeto["peso"] <= capacidad_restante:
        seleccionados.append({
            "objeto": objeto["nombre"],
            "fraccion": 1,
            "peso_tomado": objeto["peso"],
            "valor_obtenido": objeto["valor"]
        })

        valor_total += objeto["valor"]
        capacidad_restante -= objeto["peso"]

    # Si no entra completo, se toma solo una fracción
    else:
        fraccion = capacidad_restante / objeto["peso"]
        valor_obtenido = objeto["valor"] * fraccion

        seleccionados.append({
            "objeto": objeto["nombre"],
            "fraccion": fraccion,
            "peso_tomado": capacidad_restante,
            "valor_obtenido": valor_obtenido
        })

        valor_total += valor_obtenido
        capacidad_restante = 0

return valor_total, seleccionados

# Datos de prueba
objetos = [
    {"nombre": "A", "valor": 60, "peso": 10},
    {"nombre": "B", "valor": 100, "peso": 20},
    {"nombre": "C", "valor": 120, "peso": 30}
]

capacidad = 50

valor_total, seleccionados = mochila_fraccionaria(objetos, capacidad)

print("Objetos seleccionados:")
for item in seleccionados:
    print(item)

print("\nValor total obtenido:", valor_total)

```

Objetos seleccionados:

{'objeto': 'A', 'fraccion': 1, 'peso_tomado': 10, 'valor_obtenido': 60}

{'objeto': 'B', 'fraccion': 1, 'peso_tomado': 20, 'valor_obtenido': 100}

{'objeto': 'C', 'fraccion': 0.6666666666666666, 'peso_tomado': 20, 'valor_obtenido': 80.0}

Valor total obtenido: 240.0

1.3 Análisis

El algoritmo selecciona primero los objetos con mayor relación valor/peso.

En el ejemplo, los objetos se ordenan así:

1. Objeto A: $60 / 10 = 6$
2. Objeto B: $100 / 20 = 5$
3. Objeto C: $120 / 30 = 4$

Como la mochila tiene capacidad 50, se toma el objeto A completo, el objeto B completo y una parte del objeto C.

La complejidad principal está en ordenar los objetos, por lo que el algoritmo tiene una complejidad aproximada de:

$$O(n \log n)$$

2. Backtracking: Resolución de Sudoku

2.1 Descripción del problema

El Sudoku es un tablero de 9x9 donde se deben completar las celdas vacías con números del 1 al 9, cumpliendo estas reglas:

- No se puede repetir un número en una misma fila.
- No se puede repetir un número en una misma columna.
- No se puede repetir un número en una subcuadrícula de 3x3.

2.2 Idea del algoritmo

El algoritmo utiliza **backtracking**:

1. Busca una celda vacía.
2. Prueba números del 1 al 9.
3. Verifica si el número cumple las reglas del Sudoku.
4. Si es válido, continúa con la siguiente celda.
5. Si más adelante no se puede continuar, retrocede y prueba otro número.

Es backtracking porque construye una solución paso a paso y descarta caminos que no llevan a una solución válida.

```
In [7]: def imprimir_tablero(tablero):
        """Imprime el tablero de Sudoku con un formato legible."""
        for fila in range(9):
            if fila % 3 == 0 and fila != 0:
                print("-" * 21)

            for columna in range(9):
                if columna % 3 == 0 and columna != 0:
                    print("|", end=" ")

                print(tablero[fila][columna] if tablero[fila][columna] != 0 else ".", end=" ")

            print()

def buscar_celda_vacia(tablero):
    """Busca la primera celda vacía del tablero. Las celdas vacías se representan con 0."""
    for fila in range(9):
        for columna in range(9):
            if tablero[fila][columna] == 0:
                return fila, columna

    return None

def es_valido(tablero, numero, fila, columna):
    """Verifica si un número puede colocarse en una posición determinada."""

    # Validar fila
    if numero in tablero[fila]:
        return False

    # Validar columna
    for i in range(9):
        if tablero[i][columna] == numero:
            return False

    # Validar subcuadrícula 3x3
    inicio_fila = (fila // 3) * 3
    inicio_columna = (columna // 3) * 3

    for i in range(inicio_fila, inicio_fila + 3):
        for j in range(inicio_columna, inicio_columna + 3):
            if tablero[i][j] == numero:
                return False

    return True

def resolver_sudoku(tablero):
    """Resuelve el Sudoku utilizando backtracking."""

    celda_vacia = buscar_celda_vacia(tablero)

    # Si no hay celdas vacías, el Sudoku está resuelto
    if not celda_vacia:
        return True

    fila, columna = celda_vacia

    # Probar números del 1 al 9
    for numero in range(1, 10):
        if es_valido(tablero, numero, fila, columna):
            tablero[fila][columna] = numero

            # Intentar resolver el resto del tablero
            if resolver_sudoku(tablero):
```



```
        return True

        # Si no funcionó, se deshace la decisión
        tablero[fila][columna] = 0

    return False

# Sudoku de prueba
sudoku = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],

    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],

    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

print("Sudoku inicial:")
imprimir_tablero(sudoku)

if resolver_sudoku(sudoku):
    print("\nSudoku resuelto:")
    imprimir_tablero(sudoku)
else:
    print("No se encontró solución.")
```

```
Sudoku inicial:
5 3 . | . 7 . | . . .
6 . . | 1 9 5 | . . .
. 9 8 | . . . | . 6 .
-----
8 . . | . 6 . | . . 3
4 . . | 8 . 3 | . . 1
7 . . | . 2 . | . . 6
-----
. 6 . | . . . | 2 8 .
. . . | 4 1 9 | . . 5
. . . | . 8 . | . 7 9
```

```
Sudoku resuelto:
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
-----
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
-----
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

2.3 Análisis

El algoritmo prueba posibles valores en cada celda vacía y continúa solamente si la solución parcial es válida.

Cuando encuentra un conflicto, vuelve atrás y cambia la decisión anterior.

Esa es la característica principal del **backtracking**.

En el peor caso, el algoritmo puede probar muchas combinaciones, por lo que su complejidad es alta. Sin embargo, las restricciones del Sudoku permiten descartar muchas opciones inválidas durante el proceso.

3. Divide y vencerás: Merge Sort

3.1 Descripción del problema

Merge Sort es un algoritmo de ordenamiento que permite ordenar una lista de elementos.

3.2 Idea del algoritmo

La técnica de **divide y vencerás** consiste en:

- 1. Dividir el problema en subproblemas más pequeños.
- 2. Resolver cada subproblema de forma recursiva.
- 3. Combinar las soluciones parciales.

En Merge Sort, la lista se divide en dos mitades hasta llegar a listas de un solo elemento. Luego, esas listas se combinan de forma ordenada.

```
In [8]: def merge_sort(lista):
        """Ordena una lista usando el algoritmo Merge Sort."""

        # Caso base: una lista vacía o de un elemento ya está ordenada
        if len(lista) <= 1:
            return lista

        mitad = len(lista) // 2

        # Dividir la lista en dos partes
        izquierda = lista[:mitad]
        derecha = lista[mitad:]

        # Ordenar cada parte de forma recursiva
        izquierda_ordenada = merge_sort(izquierda)
        derecha_ordenada = merge_sort(derecha)

        # Combinar ambas partes ordenadas
        return combinar(izquierda_ordenada, derecha_ordenada)

def combinar(izquierda, derecha):
    """Combina dos listas ordenadas en una sola lista ordenada."""

    resultado = []
    i = 0
    j = 0

    while i < len(izquierda) and j < len(derecha):
        if izquierda[i] <= derecha[j]:
            resultado.append(izquierda[i])
            i += 1
        else:
            resultado.append(derecha[j])
            j += 1

    # Agregar los elementos restantes
    resultado.extend(izquierda[i:])
    resultado.extend(derecha[j:])

    return resultado

# Lista de prueba
lista = [38, 27, 43, 3, 9, 82, 10]

print("Lista original:", lista)

lista_ordenada = merge_sort(lista)

print("Lista ordenada:", lista_ordenada)
```

Lista original: [38, 27, 43, 3, 9, 82, 10]
Lista ordenada: [3, 9, 10, 27, 38, 43, 82]

3.3 Análisis

Merge Sort divide la lista en mitades hasta llegar a listas pequeñas. Luego combina esas partes de forma ordenada.

La complejidad de Merge Sort es:

$O(n \log n)$

Esto ocurre porque la lista se divide repetidamente en dos partes y, en cada nivel de división, se recorren los elementos para combinarlos.

4. Comparación de técnicas

Técnica	Algoritmo utilizado	Estrategia principal	Tipo de problema
Voraz	Mochila fraccionaria	Elegir la mejor opción local en cada paso	Optimización
Backtracking	Sudoku	Probar soluciones y retroceder ante conflictos	Búsqueda de solución válida

Técnica	Algoritmo utilizado	Estrategia principal	Tipo de problema
Divide y vencerás	Merge Sort	Dividir, resolver y combinar	Ordenamiento

Conclusión

En esta actividad se implementaron tres técnicas diferentes de diseño algorítmico.

La **mochila fraccionaria** muestra cómo una estrategia voraz puede resolver un problema de optimización cuando se permite tomar fracciones de objetos.

El **Sudoku** demuestra el uso de backtracking, ya que el algoritmo prueba valores posibles y retrocede cuando encuentra una contradicción. Finalmente, **Merge Sort** aplica divide y vencerás al separar una lista en partes más pequeñas, resolverlas y combinarlas ordenadamente.

Cada técnica tiene una forma distinta de abordar los problemas, por lo que es importante reconocer cuándo conviene utilizar cada una.